

Instruction Set Design

In designing an instruction set, the designer has some parameters to consider:

Instruction Length: Longer instructions give more flexibility, at the cost of storage, complexity

Number of operations: In principle, a computer requires one instruction.
(See, for example, SUBLEQ A,B,C

Writing programs for such a machine is a challenge

Instruction Set Design

- **Number of operations:** Most processors provide at least a minimum set of operations (LD, ST, arithmetic, branch)
 - More complex operations can be made of simpler instructions (in software) or provided (in hardware).
 - Having more specialized instructions **can** lead to shorter programs (but doesn't always)
 - However, a larger instruction set means longer instructions, wider buses and more memory

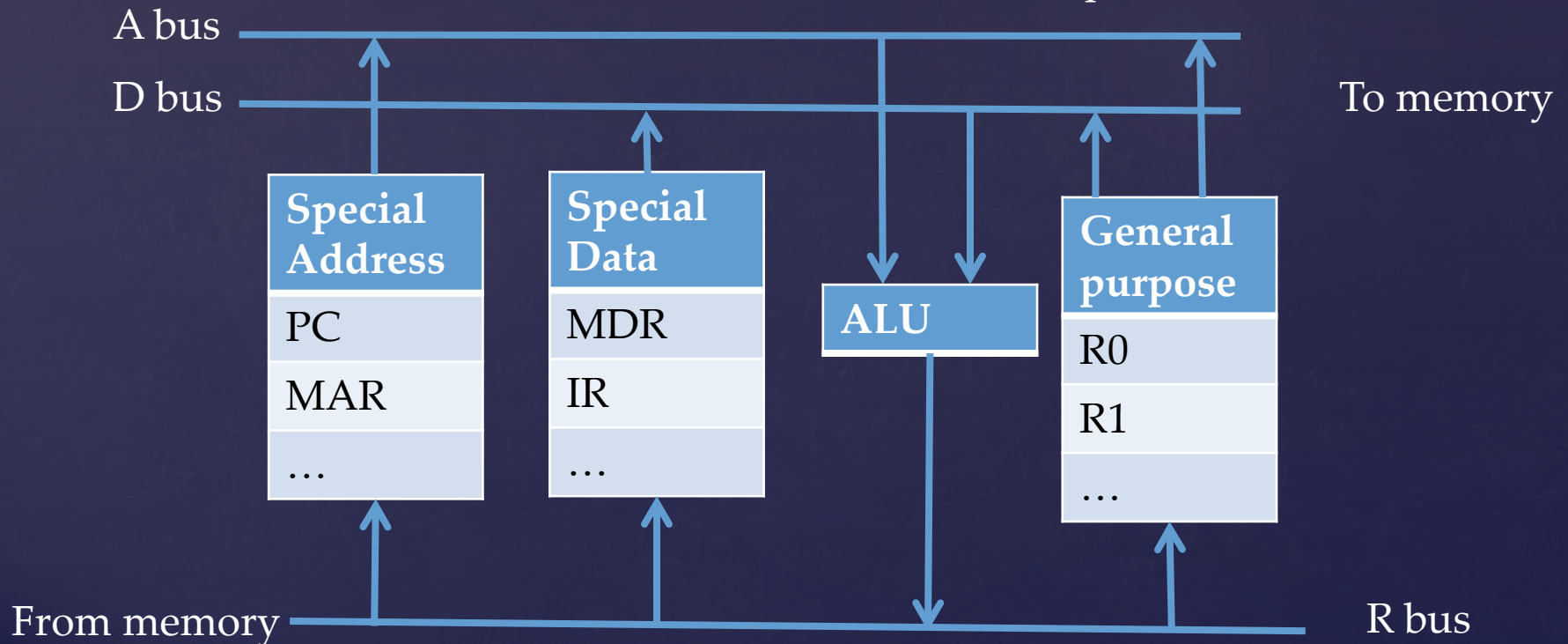
Instruction Set Design

Addressing Modes: Many instruction sets offer different **Addressing Modes**.

Instruction	Meaning
LD R, # imm	Put value imm into R. imm in code
LD R, \$ addr	Put value from [addr] into R. addr in code
LD R1, R2	Put value from [R2] into R1.
LD R1, R2+	Put value from [R2] into R1 then increment R2
LD R1, R2, R3	Put value from [R2+R3] into R1
LD R1, R2, # offs	Put value from [R2+ offs] into R1. offs in code
PUSH R1	Put R1 on stack. Stack pointer is implied

Addressing modes (cont).

From the point of view of the datapath, these might as well all be different instructions. Let's look at what is required.

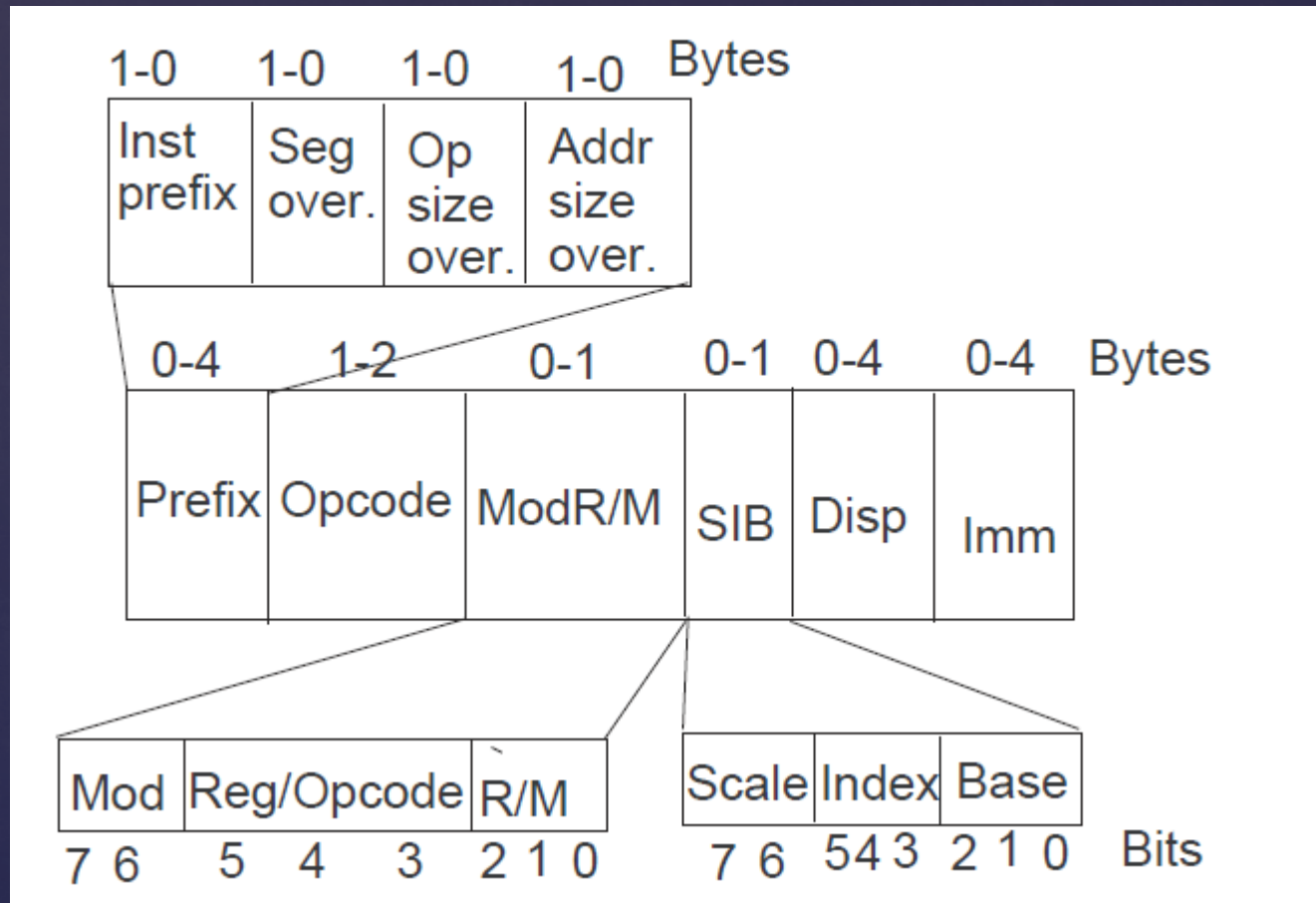


Instruction Set Design

Layout: Each instruction needs to specify the following information.

Instruction	Meaning
Opcode	What operation is being performed
Addressing mode	Sometimes requires extra bits, sometimes in opcode
Operands	Immediate operands? Register operands? Memory address operands? Size matters.
Granularity	Some machines distinguish byte operations from word operations or long operations

Intel Pentium Instructions



Intel Pentium Instructions

Element	Meaning
Prefix (opt)	LOCK memory (for shared processor environments) or REP (repeat) this operation on multiple elements
Seg ov (opt)	Specifies the segment for this instruction, overrides default
Addr size (opt)	Specifies the size of the address
Operand size (opt)	Specifies the size of operands
Opcode (req)	What is the operation. Unchanged since 1980
ModR/M (opt)	Specifies the addressing mode
SIB (opt)	For some addressing modes, specify scale, index, base
Displacement (opt)	Used by some addressing modes
Immediate (opt)	Can be 8-, 16-, 32- bits

T4

Many years ago, for another course, I designed the T4. Every T4 instruction looks like this:

Opcode	Ri	Rj	Rk
15..12	11..8	7..4	3..0

The T4 has only 16 possible operations. With one exception, they all have the form:

OP Ri, Rj, Rk

S4

Some years after that, for still another course, I designed the S4. The S4 had the following instruction set:

Type	Syntax	Length	High byte	Low byte	Comment
Byte imm	OP 	1	Opcode		
Word imm	OP <W>	2	Opcode	X	<W> in second word
Inherent	OP	1	Opcode	X	

The S4 had only a single accumulator and used the stack for each operation. Eight bits of opcode meant that 256 operations were possible